MTH3045: Statistical Computing

Dr. Ben Youngman
b.youngman@exeter.ac.uk
Laver 817; ext. 2314

23/3/2026

Week 11 lecture 1

Quasi-Newton methods I

- Between Newton's method and steepest descent lie **quasi-Newton** methods
- These essentially employ Newton's method, but with some approximation to the Hessian matrix
- Instead of the Newton's method search direction

$$\mathbf{p}_i = -\left[\nabla^2 f(\boldsymbol{\theta}_i)\right]^{-1} \nabla f(\boldsymbol{\theta}_i)$$

consider the search direction

$$\tilde{\mathbf{p}}_i = -\mathbf{H}_i^{-1} \nabla f(\boldsymbol{\theta}_i)$$

where $\mathbf{H}_i$ is an approximation to the Hessian matrix $\nabla^2 f(\boldsymbol{\theta}_i)$ at the ith iteration

Quasi-Newton methods II

- We might, for example, want to avoid explicitly calculating $\nabla^2 f(\boldsymbol{\theta}_i)$ because it's a matrix that's sufficiently more difficult to calculate than the gradient (e.g. mathematically, or just in terms of time)
  - so that using an approximation to the Hessian matrix (provided it is an adequate approximation) gives a more efficient approach to optimisation than using the Hessian matrix itself
- We should typically expect quasi-Newton methods to converge slower than Newton's method, but provided that convergence isn't too much slower or less reliable, then we may prefer this over analytically forming the Hessian matrix

BFGS (Broyden–Fletcher–Goldfarb–Shanno) I

- In MTH3045 we shall consider the so-called **BFGS** (shorthand for Broyden–Fletcher–Goldfarb–Shanno) quasi-Newton algorithm
- Put simply, at iteration i, the BFGS algorithm assumes that

$$\nabla^2 f(\boldsymbol{\theta}_i) \simeq \mathbf{H}_i = \mathbf{H}_{i-1} + \frac{\mathbf{y}_i \mathbf{y}_i^{\mathrm{T}}}{\mathbf{y}_i^{\mathrm{T}} \mathbf{s}_i} - \frac{(\mathbf{H}_{i-1})^{-1} \mathbf{s}_i \mathbf{s}_i^{\mathrm{T}} (\mathbf{H}_{i-1})^{-T}}{\mathbf{s}_i^{\mathrm{T}} (\mathbf{H}_{i-1})^{-1} \mathbf{s}_i},$$

  where $\mathbf{s}_i = \boldsymbol{\theta}_i - \boldsymbol{\theta}_{i-1}$ and $\mathbf{y}_i = \nabla f(\boldsymbol{\theta}_i) - \nabla f(\boldsymbol{\theta}_{i-1})$
- Hence the BFGS algorithm uses differences in the gradients of successive iterations to approximate the Hessian matrix
- We now note that we use $\mathbf{H}_i$ in $\mathbf{p}_i = -[\mathbf{H}_i]^{-1} \nabla f(\boldsymbol{\theta}_i)$
- We can avoid solving this system of linear equations by instead directly obtaining $[\mathbf{H}_i]^{-1}$ through

$$[\mathbf{H}_i]^{-1} = \left(\mathbf{I}_p - \frac{\mathbf{s}_i \mathbf{y}_i^{\mathsf{T}}}{\mathbf{y}_i^{\mathsf{T}} \mathbf{s}_i} \right) [\mathbf{H}_{i-1}]^{-1} \left(\mathbf{I}_p - \frac{\mathbf{y}_i \mathbf{s}_i^{\mathsf{T}}}{\mathbf{s}_i^{\mathsf{T}} \mathbf{y}_i} \right) + \frac{\mathbf{s}_i \mathbf{s}_i^{\mathsf{T}}}{\mathbf{y}_i^{\mathsf{T}} \mathbf{y}_i}$$

BFGS (Broyden–Fletcher–Goldfarb–Shanno) II

- The following R function updates $[\mathbf{H}_{i-1}]^{-1}$ to $[\mathbf{H}_i]^{-1}$ given $\boldsymbol{\theta}_i$, $\boldsymbol{\theta}_{i-1}$, $\nabla f(\boldsymbol{\theta}_{i-1})$ and $\nabla f(\boldsymbol{\theta}_i)$, which are the arguments x0, x1, g0 and g1, respectively

```r
iH1 <- function(x0, x1, g0, g1, iH0) {
  # Function to update Hessian matrix
  # x0 and x1 are p-vectors of second to last and last estimates, respectively
  # g0 and g1 are p-vectors of second to last and last gradients, respectively
  # iH0 is previous estimate of p x p Hessian matrix
  # returns a p x p matrix
  s0 <- x1 - x0
  y0 <- g1 - g0
  denom <- sum(y0 * s0)
  I <- diag(rep(1, 2))
  pre <- I - tcrossprod(s0, y0) / denom
  post <- I - tcrossprod(y0, s0) / denom
  last <- tcrossprod(s0) / denom
  pre %*% iH0 %*% post + last
}
```

Example: Weibull maximum likelihood: BFGS I

- Repeat Example 5.5 using the BFGS method
- Comment on how it compares to Newton's method

Example: Weibull maximum likelihood: BFGS II

- The following code implements five iterations of the BFGS method

```
## This build of rgl does not include OpenGL functions.  Use
##  rglwidget() to display results, e.g. via options(rgl.printRglwidget = TRUE)
```

```
iterations <- 5
xx <- matrix(0, iterations + 1, 2)
dimnames(xx) <- list(paste('iter', 0:iterations), c('lambda', 'k'))
xx[1, ] <- c(1.6, .6)
g <- iH <- list()
for (i in 2:(iterations + 1)) {
  g[[i]] <- weib_d1(xx[i - 1, ], y0, mult = -1)
  if (sqrt(sum(g[[i]]^2)) < 1e-6)
    break
  if (i == 2) {
    iH[[i]] <- diag(1, 2)
  } else {
    iH[[i]] <- iH1(xx[i - 2, ], xx[i - 1, ], g[[i - 1]], g[[i]], iH[[i - 1]])
  }
  search_dir <- -(iH[[i]] %*% g[[i]])
 alpha <- line_search(xx[i - 1, ], search_dir, weib_d0, y = y0, mult = -1)
 xx[i, ] <- xx[i - 1,] + alpha * search_dir
}
```

Example: Weibull maximum likelihood: BFGS III

- Our estimates at each iteration are

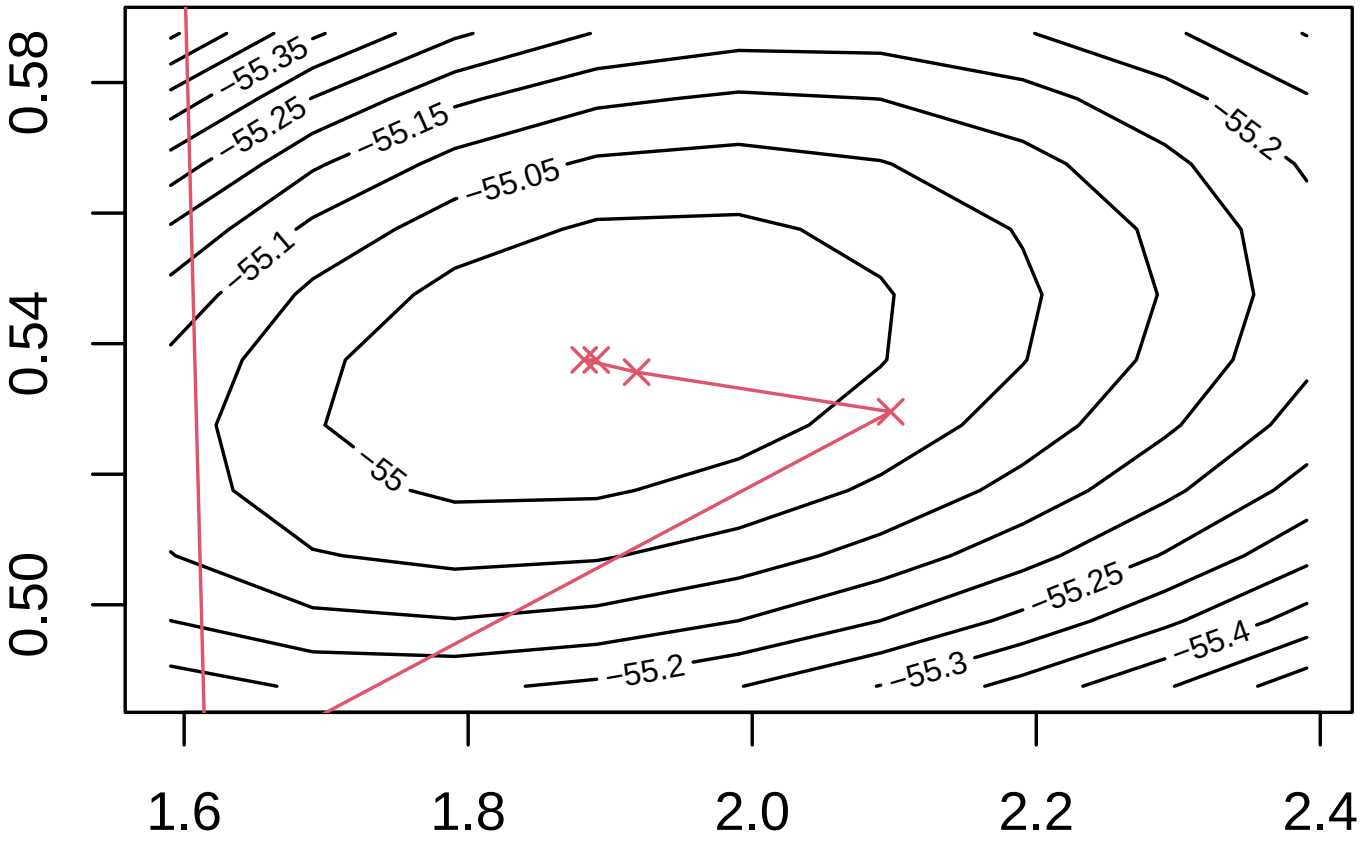
```
xx
```

```
##          lambda         k
## iter 0 1.600000 0.6000000
## iter 1 1.615241 0.4736904
## iter 2 2.097571 0.5295654
## iter 3 1.918661 0.5356343
## iter 4 1.881726 0.5375145
## iter 5 1.890167 0.5374986
```

  and we see that we need two more iterations than Newton's method to reach convergence to three decimal places

Example: Weibull maximum likelihood: BFGS IV

- Finally, we'll plot the course of the iterations



and we can see the slightly less direct route we've taken

Quasi-Newton methods in R I

- There are various options for performing quasi-Newton methods in R
- For these, we just need to supply the function to be minimised and its gradient
- The first option is to use `nlminb()` again
  - if we don't supply a function to evaluate the Hessian, then `nlminb()` uses a quasi-Newton approach
- The alternative, and possibly preferred option, is to use `optim()` with option `method = 'BFGS'`
- We'll repeat Example 5.5 using BFGS instead

Quasi-Newton methods in R II

- To use `nlminb()` we can use the following.

```r
nlminb(c(1.6, .6), weib_d0, weib_d1, y = y0, mult = -1)
```

```
## $par
## [1] 1.8900689 0.5375279
##
## $objective
## [1] 54.95316
##
## $convergence
## [1] 0
##
## $iterations
## [1] 7
##
## $evaluations
## function gradient
##        9        8
##
## $message
## [1] "relative convergence (4)"
```

Quasi-Newton methods in R III

- To use `optim()` we can use the following.

```r
optim(c(1.6, .6), weib_d0, weib_d1, y = y0, mult = -1, method = 'BFGS')
```

```
## $par
## [1] 1.8900632 0.5375283
##
## $value
## [1] 54.95316
##
## $counts
## function gradient
##       14        6
##
## $convergence
## [1] 0
##
## $message
## NULL
```

Finite-difference checking I

```r
fd <- function(x, f, delta = 1e-6, ...) {
  # Function to evaluate derivative w.r.t. vector by finite-differencing
  # x is a p-vector
  # fn is the function for which the derivative is being calculated
  # delta is the finite-differencing step, which defaults to 10^{-6}
  # returns a vector of length x
  f0 <- f(x, ...)
  p <- length(x)
  f1 <- numeric(p)
  for (i in 1:p) {
    ei <- replace(numeric(p), i, 1)
    f1[i] <- f(x + delta * ei, ...)
  }
  (as.vector(f1) - f0) / delta
}
```

Finite-difference checking II

- Finite-differencing can help us check a gradient function we've written

```r
# Check a function we've written
weib_d1(c(1.6, .6), y = y0, mult = -1)
```

```
## [1] -1.950898 16.167624
```

```r
fd(c(1.6, .6), weib_d0, y = y0, mult = -1)
```

```
## [1] -1.950895 16.167729
```

Finite-difference checking III

- Finite-differencing can also help us check whether we've reached a local minimum

```r
# Check optima
## Nelder-Mead
p_nm <- optim(c(1.6, .6), weib_d0, y = y0, mult = -1)$par
weib_d1(p_nm, y = y0, mult = -1)
```

```
## [1]  0.0004811097 -0.0116608005
```

```r
fd(p_nm, weib_d0, y = y0, mult = -1)
```

```
## [1]  0.0004823661 -0.0115609069
```

```r
## BFGS
p_bfgs <- optim(c(1.6, .6), weib_d0, weib_d1, y = y0, mult = -1, method = 'BFGS
weib_d1(p_bfgs, y = y0, mult = -1)
```

```
## [1] -1.677935e-05  1.121945e-04
```

```r
fd(p_bfgs, weib_d0, y = y0, mult = -1)
```

```
## [1] -1.552536e-05  2.120899e-04
```

Quasi-Newton methods in R IV

- We note `nlminb()` and `optim()` have essentially given the same value of `par`, i.e. for the $\hat{\lambda}$ and $\hat{k}$, which is reassuring
- Note that `nlminb()` has used fewer function evaluations than `optim()`
  - we won't go into the details of the cause of this, but it is worth noting that the functions use different stopping criteria, and slightly different variants of the BFGS algorithm
- Note also that `nlminb()` has used three more function evaluations with the BFGS method than with Newton's method
  - this is is typically the case, and reflects the improved convergence achieved by using the actual Hessian matrix with Newton's method, as opposed to the approximation that's used with the BFGS approach

Challenges I

- Go to Challenges I of the week 11 lecture 1 challenges at https://byoungman.github.io/MTH3045/challenges

Week 11 lecture 2

Nelder-Mead polytope method I

- So far we have considered derivative-based optimisation algorithms
- When we cannot analytically calculate derivatives, we can use finite-difference approximations
- However, sometimes we may want to find the minimum point on a surface for a surface that is not particularly smooth
- Then derivative information may not be helpful
- Instead, we might want an algorithm that explores a surfaces differently
- The Nelder-Mead polytope algorithm is one such approach (Nelder and Mead (1965))
- In fact, it is `R`'s default if we use `optim()`, i.e. if we don't supply `method = '...'`

Nelder-Mead polytope method II

- For the Nelder-Mead algorithm, consider $\boldsymbol{\theta} \in \mathbb{R}^p$. The algorithm starts with $p+1$ test points, $\boldsymbol{\theta}_1, \ldots, \boldsymbol{\theta}_{p+1}$, which we call *vertices*, and then proceeds as follows...

1. Compute the order statistics of $f(\boldsymbol{\theta}_1), \ldots, f(\boldsymbol{\theta}_{p+1})$ vertices, i.e. find the order
$$f(\boldsymbol{\theta}^{(1)}) \leq f(\boldsymbol{\theta}^{(2)}) \leq \ldots \leq f(\boldsymbol{\theta}^{(p+1)})$$
and check whether the termination criteria have been met (which are given later). If not, proceed to Step 2.

2. Calculate the centroid, $\boldsymbol{\theta}_o$, of $\boldsymbol{\theta}_1, \ldots, \boldsymbol{\theta}_p$, i.e. omitting $\boldsymbol{\theta}_{p+1}$, because $\boldsymbol{\theta}_{p+1}$ is the worst vertex.

3. Reflection. Compute the reflected point $\boldsymbol{\theta}_r = \boldsymbol{\theta}_o + \alpha(\boldsymbol{\theta}_o - \boldsymbol{\theta}_{p+1})$. If $f(\boldsymbol{\theta}_1) \leq f(\boldsymbol{\theta}_r) < f(\boldsymbol{\theta}_p)$, replace $\boldsymbol{\theta}_{p+1}$ with $\boldsymbol{\theta}_r$ and return to Step 1. Otherwise, proceed to Step 4.

Nelder-Mead polytope method III

4. Expansion. If $f(\boldsymbol{\theta}_r) < f(\boldsymbol{\theta}_1)$, i.e. is the best point so far, compute the expanded point $\boldsymbol{\theta}_e = \boldsymbol{\theta}_o + \gamma(\boldsymbol{\theta}_r - \boldsymbol{\theta}_o)$ for $\gamma > 1$. If $f(\boldsymbol{\theta}_e) < f(\boldsymbol{\theta}_r)$, replace $\boldsymbol{\theta}_{p+1}$ with $\boldsymbol{\theta}_r$ and return to Step 1. Otherwise, replace $\boldsymbol{\theta}_{p+1}$ with $\boldsymbol{\theta}_r$ and return to Step 1.
5. Contraction. Now $f(\boldsymbol{\theta}_r) \geq f(\boldsymbol{\theta}_p)$. Compute the contracted point $\boldsymbol{\theta}_c = \boldsymbol{\theta}_o + \rho(\boldsymbol{\theta}_{p+1} - \boldsymbol{\theta}_o)$ for $0 < \rho \leq 0.5$. If $f(\boldsymbol{\theta}_c) < f(\boldsymbol{\theta}_{p+1})$ then replace $\boldsymbol{\theta}_{p+1}$ with $\boldsymbol{\theta}_c$ and return to Step 1. Otherwise proceed to Step 6.
6. Shrink. For $i = 2, \ldots, p+1$ set $\boldsymbol{\theta}_i = \boldsymbol{\theta}_1 + \sigma(\boldsymbol{\theta}_i - \boldsymbol{\theta}_1)$ and return to Step 1.

Nelder-Mead polytope method IV

- Often the values $\alpha = 1$, $\gamma = 2$, $\rho = 0.5$ and $\sigma = 0.5$ are used
- In Step 1, termination is defined in terms of tolerances
- The main criterion is for $f(\boldsymbol{\theta}_{p+1}) - f(\boldsymbol{\theta}_1)$ to be sufficiently small, so that $f(\boldsymbol{\theta}_i)$ for $i = 1, \ldots, p+1$ are close together for all θ_i
  - hence we are hoping that *all* the $\boldsymbol{\theta}_i$ values are in the region of the true minimum
- We won't code the Nelder-Mead algorithm in `R`; instead we'll just use `optim()` and look what it does, by requesting a trace with `control = list(trace = TRUE)`

Example: Weibull maximum likelihood: Nelder-Mead I

- Use the Nelder-Mead method and `R`'s `optim()` function to find the maximum likelihood estimates of the Weibull distribution for the data of Example 5.5
- We've got everything we need for this example, i.e. the data, which we stored earlier as `y0`, and a function to evaluate the Weibull distribution's log-likelihood, `weib_d0()`
- So we just pass these to `optim()`, ensuring that `mult = -1`, so that we find the negative log-likelihood's minimum

Example: Weibull maximum likelihood: Nelder-Mead II

```r
fit_nelder <- optim(c(1.6, .6), weib_d0, y = y0, mult = -1,
                    control = list(trace = TRUE))
```

```
##   Nelder-Mead direct search function minimizer
## function value for initial parameters = 55.509247
##   Scaled convergence tolerance is 8.27152e-07
## Stepsize computed as 0.159049
## BUILD             3 60.369866 55.293827
## LO-REDUCTION      5 55.509247 55.045750
## REFLECTION        7 55.293827 55.038084
## HI-REDUCTION      9 55.045750 54.993703
## REFLECTION       11 55.038084 54.962562
## HI-REDUCTION     13 54.993703 54.961598
## ** quite a few lines suppressed **
## HI-REDUCTION     41 54.953162 54.953159
## HI-REDUCTION     43 54.953160 54.953159
## Exiting from Nelder Mead minimizer
##     45 function evaluations used
```

- The above output is telling us what `optim()` is doing as it's going along
- Specifically, `LO-REDUCTION` corresponds to a contraction, `HI-REDUCTION` to an expansion and `REFLECTION` to a reflection
- On this occasion, there was no need to shrink the simplex, which is identified as `SHRINK`

Example: Weibull maximum likelihood: Nelder-Mead III

```
fit_nelder
```

```
## $par
## [1] 1.8900970 0.5374706
##
## $value
## [1] 54.95316
##
## $counts
## function gradient
##       43       NA
##
## $convergence
## [1] 0
##
## $message
## NULL
```

- What `optim()` returns is essentially the same as we saw before for the BFGS method, i.e. roughly the same parameter estimates and function minimum, except that now there are no gradient evaluations

Example: Weibull maximum likelihood: Nelder-Mead IV

- *Remark*: The Nelder-Mead method is usually great if you're quickly looking to find a function's minimum with its gradient
  - provided it's a relatively low-dimensional function, and the function is fairly quick to evaluate
  - this is probably why it's `R`'s default
- It can be very slow for high-dimensional optimisation problems, and is also typically less reliable than gradient-based methods, except for strangely-behaved surfaces

Challenges I

- Go to Challenges I of the week 11 lecture 2 challenges at https://byoungman.github.io/MTH3045/challenges

Week 11 lecture 3

Global optimisation

- So far we have considered optimisation algorithms that usually home in on local minima, given starting points
- For multimodal surfaces, this can be undesirable
- Here we consider approaches to **global optimisation**, which are designed to find the *overall* minimum

Stochastic optimisation

- The optimisation algorithms that have been introduced so far have been deterministic
  - given a set of starting values, they will always return the same final value
- Stochastic optimisation algorithms go from one point to another probabilistically, and so will go through different sets of parameters
- We should expect them to converge to the same final value, though

Simulated annealing I

- Simulated annealing gains its name from the physical annealing process
  - a heat treatment that alters the physical and sometimes chemical properties of a material to increase its ductility and reduce its hardness, making it more workable
  - you can of course read more about it on Wikipedia[1]
- Consider a current parameter value $\boldsymbol{\theta}$ and some function we seek to minimise $f()$
- For example, $f()$ might be a negated log-likelihood
- The key to simulated annealing is that a *random* point is proposed, $\boldsymbol{\theta}^*$
  - $\boldsymbol{\theta}^*$ is drawn from some proposal density $q(\boldsymbol{\theta}^* \mid \boldsymbol{\theta})$, depends on the current value $\boldsymbol{\theta}$
- The proposal density $q()$ is chosen to be symmetric, but otherwise its choice is arbitrary

[1] https://en.wikipedia.org/wiki/Annealing_(materials_science)

Simulated annealing II

- The main aspect of simulated annealing is to work with the function

$$\pi_T(\boldsymbol{\theta}) = \exp\{-f(\boldsymbol{\theta})/T\}$$

  for some *temperature* T

- We note that as $T \searrow 0$, $\pi_T(\boldsymbol{\theta}) \to \exp\{-f(\boldsymbol{\theta})\}$
- Put algorithmically, simulated annealing works as follows

1. Propose $\boldsymbol{\theta}$ from $q(\boldsymbol{\theta}^* \mid \boldsymbol{\theta})$.
2. Generate $U \sim \text{Uniform}[0, 1]$.
3. Calculate

$$\begin{aligned}\alpha(\boldsymbol{\theta}^* \mid \boldsymbol{\theta}) &= \min\left\{\frac{\exp\left[-f(\boldsymbol{\theta}^*)/T\right]}{\exp\left[-f(\boldsymbol{\theta})/T\right]}, 1\right\} \\ &= \min\left(\exp\left\{-\left[f(\boldsymbol{\theta}^*) - f(\boldsymbol{\theta})\right]/T\right\}, 1\right).\end{aligned}$$

4. Accept $\boldsymbol{\theta}^*$ if $\alpha(\boldsymbol{\theta}^* \mid \boldsymbol{\theta}) > U$; otherwise keep $\boldsymbol{\theta}$.
5. Decrease T.

Simulated annealing III

- It is worth noting that steps 1 to 4 implement a special case of the Metropolis-Hastings algorithm for symmetric $q()$
- This algorithm is a heavily used in statistics, especially Bayesian statistics, to sample from posterior densities that have do not have or have unwieldy closed forms, typically as part of Markov chain Monte Carlo (MCMC) sampling.
- *Remark*: `R`'s default is to use a Gaussian distribution for $q()$ and the temperature at iteration i, T_i, is chosen according to $T_i = T_1 / \log\{t_{\max} \lfloor (i-1)/t_{\max} \rfloor + \exp(1)\}$ with $T_1 = t_{\max} = 10$ the default values

Example: Weibull maximum likelihood: Simulated annealing I

- Write a function to update the simulated annealing temperature according to R's rule and another function to generate Gaussian proposals with standard deviation 0.1
- Then use simulated annealing to repeat Example 5.5 with $N = 1000$ iterations and plot λ_i and k_i at each iteration using initial temperatures of $T_1 = 10$, 1 and 0.1
- The following function, `update_T()`, updates the temperature according to R's rule

```
update_T <- function(i, t0 = 10, t1 = 10) {
  # Function to update simulated annealing temperature
  # i is an integer giving the current iteration
  # t0 is a scalar giving the initial temperature
  # t1 is a integer giving how many iterations of each temperature to use
  # returns a scalar
  t0 / log(((i - 1) %/% t1) * t1 + exp(1))
}
```

Example: Weibull maximum likelihood: Simulated annealing II

- Then the following function, `q_fn()`, generates Gaussian proposals with standard deviation 0.1

```
q_fn <- function(x) {
  # Function to generate Gaussian proposals with standard deviation 0.1
  # x is the Gaussian mean as either a scalar or vector
  # returns a scalar or vector, as x
  rnorm(length(x), x, .1)
}
```

- The following function can perform simulated annealing. You won't be asked to write such a function for MTH3045, but it's illuminating to see how such a function can be written.

```
sa <- function(p0, h, N, q, T1, ...) {
# Function to perform simulated annealing
# p0 p-vector of initial parameters
# h() function to be minimised
# N number of iterations
# q proposal function
# T1 initial temperature
# ... arguments to pass to h()
# returns p x N matrix of parameter estimates at each iteration
# commands suppressed
}
```

Example: Weibull maximum likelihood: Simulated annealing III

```
sa <- function(p0, h, N, q, T1, ...) {
# comments suppressed
out <- matrix(0, N, length(p0)) # matrix to store estimates at each iteration
out[1, ] <- p0 # fill first row with initial parameter estimates
for (i in 2:N) { # N iterations
  T <- update_T(i, T1) # update temperature
  U <- runif(1) # generate U
  out[i, ] <- out[i - 1,] # carry over last parameter estimate, by default
  proposal <- q(out[i - 1,]) # generate proposal
  if (min(proposal) >= 0) { # ensure proposal valid
    h0 <- h(out[i - 1, ], ...) # evaluate h for current theta
    h1 <- h(proposal, ...) # evaluate h for proposed theta
    alpha <- min(exp(- (h1 - h0) / T), 1) # calculate M-H ratio
    if (alpha >= U) # accept if ratio sufficiently high
      out[i, ] <- proposal # swap last with proposal
  }
}
out # return all parameter estimates
}
```

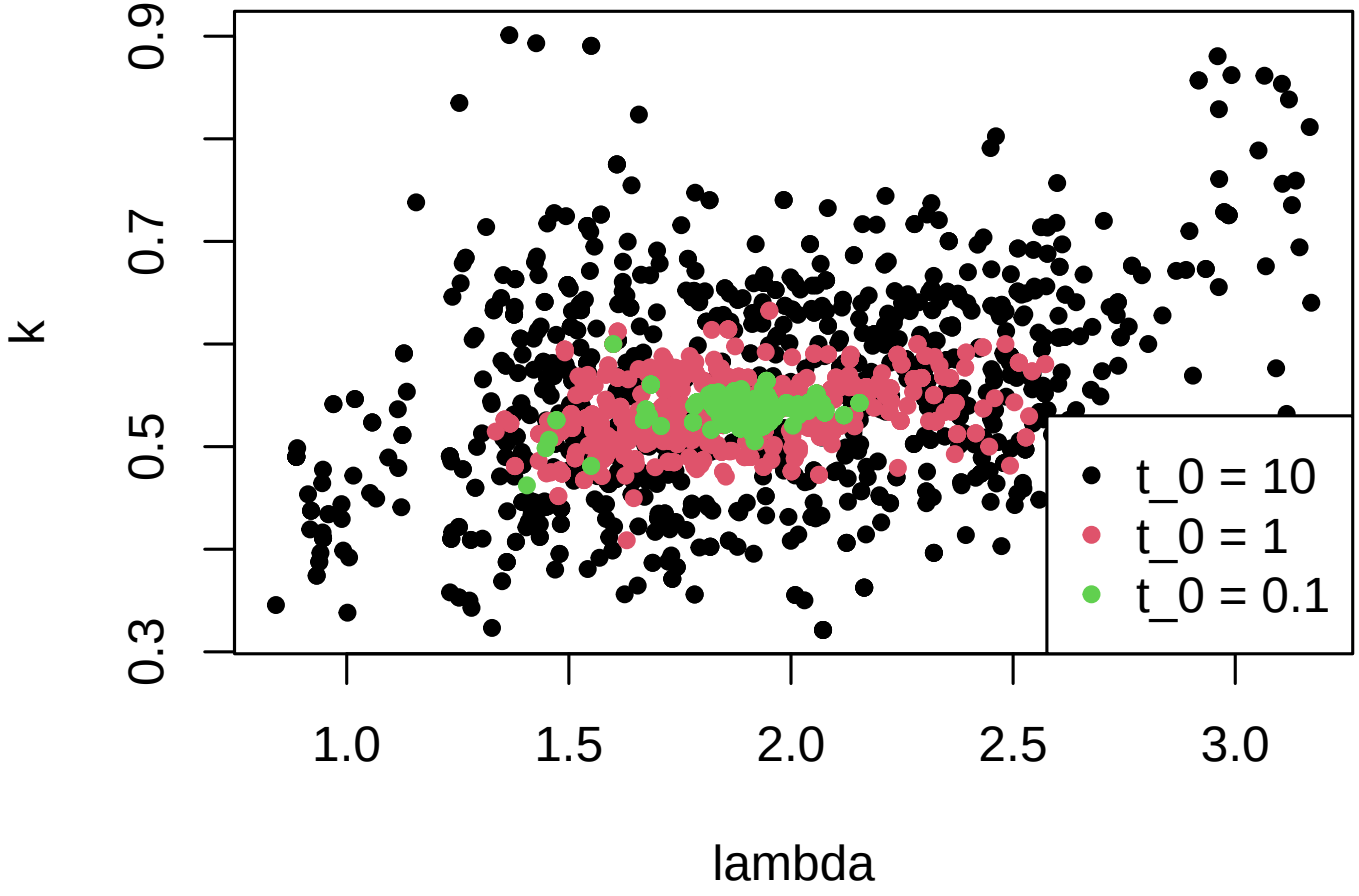
Example: Weibull maximum likelihood: Simulated annealing IV

- Then we'll specify out initial temperatures as `T_vals`, and loop over these with `sa()`...

```
# values to use for initial temperature
T_vals <- c(10, 1, .1)
# loop over values, and plot
for (j in 1:length(T_vals)) {
  T1 <- T_vals[j]
  sa_result <- sa(c(1.6, .6), weib_d0, 1e3, q_fn, T1, y = y0, mult = -1)
  if (j == 1) {
    plot(sa_result, col = j, pch = 20, xlab = 'lambda', ylab = 'k')
  } else {
    points(sa_result, col = j, pch = 20)
  }
}
legend('bottomright', pch = 20, col = 1:length(T_vals),
       legend = paste("t_0 =", T_vals), bg = 'white')
```

Example: Weibull maximum likelihood: Simulated annealing V

- . . . and then plot the resulting parameter estimates for each temperature and each iteration



- We see that lower initial temperatures bring smaller clouds of parameter estimates.

Simulated annealing in R I

- Simulated annealing is built in to R's `optim()` function and requires `method = 'SANN'`
- **Example**: Repeat Example 5.10 using R's `optim()` function to perform simulated annealing
- Report the best value of the objective function every 100 iterations

Simulated annealing in R II

- We'll use the following call

```r
optim(c(1.6, .6), weib_d0, y = y0, mult = -1, method = 'SANN',
      control = list(trace = 1, REPORT = 10, maxit = 1e3))
```

```
## sann objective function values
## initial       value 55.677933
## iter      100 value 54.956777
## iter      200 value 54.956777
## iter      300 value 54.956777
## iter      400 value 54.956777
## iter      500 value 54.956777
## iter      600 value 54.956433
## iter      700 value 54.956433
## iter      800 value 54.955008
## iter      900 value 54.954558
## iter      999 value 54.954558
## final         value 54.954558
## sann stopped after 999 iterations
```

```
## $par
## [1] 1.9256449 0.5387653
##
## $value
## [1] 54.95456
##
## $counts
```

Simulated annealing in R III

```
optim(c(1.6, .6), weib_d0, y = y0, mult = -1, method = 'SANN',
      control = list(maxit = 1e3))[1]
```

```
## $par
## [1] 1.7984842 0.5899355
```

- The parameter estimates are some way off those from earlier
- With more iterations, simulated annealing would gradually get closer to the true minimum
- Note that the `control$REPORT` argument specifies the frequency in terms of how often the temperature changes; so R reports the status of the optimiser each (`control$tmax * control$REPORT`)th iteration, noting that `control$tmax` defaults to 10.
- *Remark*: It's sometimes a good tactic to use simulated annealing to get close to the minimum, and then to employ one of the previously discussed deterministic optimisation methods to get a more accurate estimate
- This is especially useful if we're unsure whether we're starting off with sensible initial parameter estimates

Bibliographic notes

- By far the best resource for reach up on numerical optimisation is Nocedal and Wright (2006)
  - Chapter 3 covers Newton's method and line search
  - Chapter 6 covers quasi-Newton methods
  - Chapter 8 covers derivative-free optimisation, including the Nelder-Method in Section 9.5
- Optimisation is also covered in Monahan (2011, chap. 8) and in Wood (2015, sec. 5.1)
- Simulated annealing is covered in Press et al. (2007, sec. 10.12)
- Root-finding is covered in Monahan (2011, sec. 8.3) and Press et al. (2007, chap. 9)

Bibliographic notes

- By far the best resource for reach up on numerical optimisation is Nocedal and Wright (2006)
  - Chapter 3 covers Newton's method and line search
  - Chapter 6 covers quasi-Newton methods
  - Chapter 8 covers derivative-free optimisation, including the Nelder-Method in Section 9.5
- Optimisation is also covered in Monahan (2011, chap. 8) and in Wood (2015, sec. 5.1)
- Simulated annealing is covered in Press et al. (2007, sec. 10.12)
- Root-finding is covered in Monahan (2011, sec. 8.3) and Press et al. (2007, chap. 9)

Challenges I

- Go to Challenges I of the week 11 lecture 3 challenges at https://byoungman.github.io/MTH3045/challenges

Exam tips

1. If you're unsure whether your derivative function is correct, why not use finite-differencing to check it? Having generic finite-differencing functions to hand, such as `fd()` in the lecture notes, could be very helpful. Finite-differencing can also be used to approximate Hessian matrices.
2. Balance correcting code with moving on to the next question. If your code has a small error that's causing it to not run properly, then you may still pick up marks for a partially correct answer.

References

Monahan, John F. 2011. *Numerical Methods of Statistics*. 2nd ed. Cambridge University Press. https://doi.org/10.1017/CBO9780511977176.

Nelder, J. A., and R. Mead. 1965. "A Simplex Method for Function Minimization." *The Computer Journal* 7 (4): 308–13. https://doi.org/10.1093/comjnl/7.4.308.

Nocedal, J., and S. Wright. 2006. *Numerical Optimization*. 2nd ed. Springer Series in Operations Research and Financial Engineering. Springer New York. https://books.google.co.uk/books?id=VbHYoSyelFcC.

Press, W. H., S. A. Teukolsky, W. T. Vetterling, and B. P. Flannery. 2007. *Numerical Recipes: The Art of Scientific Computing*. 3rd ed. Cambridge University Press. https://books.google.co.uk/books?id=1aAOdzK3FegC.

Wood, Simon N. 2015. *Core Statistics*. Institute of Mathematical Statistics Textbooks. Cambridge University Press. https://doi.org/10.1017/CBO9781107741973.